

Cluster Autoscaler in EKS

1. What is Cluster Autoscaler?

The **Cluster Autoscaler** is a Kubernetes component that automatically adjusts the number of nodes in your cluster.

It does two things:

- **➕ Scale UP:** Adds nodes when pods are pending (not enough resources)
- **➖ Scale DOWN:** Removes unused nodes when they are underutilized

It works with your cloud provider's node groups (in EKS: Auto Scaling Groups).

2. Why do we need it?

Without autoscaling:

- Pods stay in **Pending state** when nodes are full
- You manually increase EC2 instances
- You waste money running idle nodes 24/7

With Cluster Autoscaler:

- Your cluster becomes **self-healing + elastic**
 - Handles traffic spikes automatically
 - Reduces cost by scaling down unused nodes
-

3. Where it fits in EKS architecture

In Kubernetes on Amazon Elastic Kubernetes Service (EKS):

- Worker nodes = EC2 instances in Auto Scaling Group
- Cluster Autoscaler watches:
 - Pending pods

- Node capacity
- Then calls AWS APIs to scale ASG

4. What is Pod Autoscaler?

Pod Autoscaler usually refers to **Horizontal Pod Autoscaler (HPA)** in Kubernetes.

It automatically adjusts the **number of pods** in a deployment based on metrics like:

- CPU usage
- Memory usage
- Custom metrics (requests/sec, latency, etc.)

In simple terms:

“If load increases → add more pods

If load decreases → remove pods”

Example (HPA behavior)

You deploy a web app:

- 2 pods running normally
- Traffic increases
- CPU hits 80%
- HPA increases pods to 10

```
Scale PODS: 2 → 10
```

5. Key Difference (Very Important)

Feature	Pod Autoscaler (HPA)	Cluster Autoscaler
Scales	Pods	Nodes (EC2 instances)
Level	Application level	Infrastructure level
Trigger	CPU / memory / metrics	Pending pods / unschedulable pods
Works on	Deployment/ReplicaSet	Node Groups (ASG in EKS)
Goal	Handle app load	Provide capacity for pods

6. LAB OVERVIEW

We will:

1. Create EKS cluster with managed node group
2. Enable IAM OIDC provider
3. Attach IAM policy for autoscaler
4. Deploy Cluster Autoscaler using Helm
5. Test scale-up (by creating load)
6. Observe scale-down

PREREQUISITES

Make sure you have:

- `kubectl`
- `eksctl`
- `aws cli`
- `helm`

Check:

```
kubectl version --client
eksctl version
```

```
aws --version
helm version
```

```
ubuntu@ip-172-31-38-241:~$ kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1
ubuntu@ip-172-31-38-241:~$ eksctl version
0.227.0
ubuntu@ip-172-31-38-241:~$ aws --version
aws-cli/2.34.63 Python/3.14.5 Linux/7.0.0-1004-aws exe/x86_64.ubuntu.26
ubuntu@ip-172-31-38-241:~$ helm version
version.BuildInfo{Version:"v4.2.0", GitCommit:"06468084e85c244c712834933d25ea232a4c2093", GitTreeState:"clean", GoVersion:"go1.26.3", KubeClientVer
sion:"v1.36"}
ubuntu@ip-172-31-38-241:~$
```

Create EKS Cluster

```
eksctl create cluster \
  --name=live-eks-cluster \
  --region=us-east-1 \
  --zones=us-east-1a,us-east-1b \
  --nodegroup-name=standard-nodes \
  --node-type=t3a.medium \
  --nodes=2 \
  --nodes-min=2 \
  --nodes-max=10 \
  --managed
```

Enable OIDC

Cluster Autoscaler uses IAM Roles for Service Accounts (IRSA)

```
eksctl utils associate-iam-oidc-provider \
  --region us-east-1 \
  --cluster live-eks-cluster \
  --approve
```

```
ubuntu@ip-172-31-38-241:~$ eksctl utils associate-iam-oidc-provider \
  --region us-east-1 \
  --cluster live-eks-cluster \
  --approve
2026-06-09 10:32:36 [i] will create IAM Open ID Connect provider for cluster "live-eks-cluster" in "us-east-1"
2026-06-09 10:32:36 [✓] created IAM Open ID Connect provider for cluster "live-eks-cluster" in "us-east-1"
ubuntu@ip-172-31-38-241:~$
```

Create IAM Policy for Autoscaler

Create policy:

```
cat<<EOF > cluster-autoscaler-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "autoscaling:DescribeAutoScalingGroups",
        "autoscaling:DescribeAutoScalingInstances",
        "autoscaling:DescribeLaunchConfigurations",
        "autoscaling:DescribeTags",
        "autoscaling:SetDesiredCapacity",
        "autoscaling:TerminateInstanceInAutoScalingGroup",
        "ec2:DescribeLaunchTemplateVersions"
      ],
      "Resource": "*"
    }
  ]
}
EOF
```

Create IAM policy:

```
aws iam create-policy \
--policy-name AmazonEKSClusterAutoscalerPolicy \
--policy-document file://cluster-autoscaler-policy.json
```

```

"autoscaling:DescribeTags",
"autoscaling:SetDesiredCapacity",
"autoscaling:TerminateInstanceInAutoScalingGroup",
"ec2:DescribeLaunchTemplateVersions"
],
"Resource": "*"
}
}
]
EOF
ubuntu@ip-172-31-38-241:~$ ls
2-tier-todo-app  aws  awscli.v2.zip  cluster-autoscaler-policy.json
ubuntu@ip-172-31-38-241:~$ aws iam create-policy \
--policy-name AmazonEKSClusterAutoscalerPolicy \
--policy-document file://cluster-autoscaler-policy.json

-----
CreatePolicy
-----
Policy
-----
Arn | arn:aws:iam::551322107788:policy/AmazonEKSClusterAutoscalerPolicy
AttachmentCount | 0
CreateDate | 2026-06-09T10:34:07+00:00
DefaultVersionId | v1
IsAttachable | True
Path | /
PermissionsBoundaryUsageCount | 0
PolicyId | ANPAYAXK3X6GEUNBUE225
PolicyName | AmazonEKSClusterAutoscalerPolicy
UpdateDate | 2026-06-09T10:34:07+00:00
-----

```

Save the **Policy ARN**.

```
arn:aws:iam::551322107788:policy/AmazonEKSClusterAutoscalerPo
licy
```

Create Service Account with IAM Role

Replace:

- `<ACCOUNT_ID>`
- `<POLICY_ARN>`

```
eksctl create iamserviceaccount \
--cluster live-eks-cluster \
--namespace kube-system \
--name cluster-autoscaler \
--attach-policy-arn arn:aws:iam::<ACCOUNT_ID>:policy/AmazonEK
SClusterAutoscalerPolicy \
--approve \
--override-existing-serviceaccounts
```

```

ubuntu@ip-172-31-38-241:~$ eksctl create iamserviceaccount \
--cluster live-eks-cluster \
--namespace kube-system \
--name cluster-autoscaler \
--attach-policy-arn arn:aws:iam::551322107788:policy/AmazonEKSClusterAutoscalerPolicy \
--approve \
--override-existing-serviceaccounts
2026-06-09 10:36:46 [i] 1 iamserviceaccount (kube-system/cluster-autoscaler) was included (based on the include/exclude rules)
2026-06-09 10:36:46 [!] metadata of serviceaccounts that exist in Kubernetes will be updated, as --override-existing-serviceaccounts was set
2026-06-09 10:36:46 [i] 1 task: {
  2 sequential sub-tasks: {
    create IAM role for serviceaccount "kube-system/cluster-autoscaler",
    create serviceaccount "kube-system/cluster-autoscaler",
  }
}
2026-06-09 10:36:46 [i] building iamserviceaccount stack "eksctl-live-eks-cluster-addon-iamserviceaccount-kube-system-cluster-autoscaler"
2026-06-09 10:36:46 [i] deploying stack "eksctl-live-eks-cluster-addon-iamserviceaccount-kube-system-cluster-autoscaler"
2026-06-09 10:36:46 [i] waiting for CloudFormation stack "eksctl-live-eks-cluster-addon-iamserviceaccount-kube-system-cluster-autoscaler"
2026-06-09 10:37:17 [i] waiting for CloudFormation stack "eksctl-live-eks-cluster-addon-iamserviceaccount-kube-system-cluster-autoscaler"
2026-06-09 10:37:17 [i] created serviceaccount "kube-system/cluster-autoscaler"
ubuntu@ip-172-31-38-241:~$

```

Install Cluster Autoscaler using Helm

You can take the helm chart from [artificathub.io](https://artifacthub.io/packages/helm/cluster-autoscaler/cluster-autoscaler)

The screenshot shows the Artifact Hub interface for the 'cluster-autoscaler' Helm chart. The chart is described as 'Scales Kubernetes worker nodes within autoscaling groups.' The page includes a search bar, navigation links (DOCS, STATS, SIGN UP, SIGN IN), and a sidebar with 'INSTALL', 'TEMPLATES', 'DEFAULT VALUES', and 'CHANGELOG' buttons. The main content area displays the chart name and a 'TL;DR' section with the following instructions:

```

$ helm repo add autoscaler https://kubernetes.github.io/autoscaler

# Method 1 - Using Autodiscovery
$ helm install my-release autoscaler/cluster-autoscaler \
--set 'autoDiscovery.clusterName'=<CLUSTER NAME>

# Method 2 - Specifying groups manually
$ helm install my-release autoscaler/cluster-autoscaler \
--set "autoscalingGroups[0].name=your-asg-name" \
--set "autoscalingGroups[0].maxSize=10" \
--set "autoscalingGroups[0].minSize=1"

```

Add repo:

```

helm repo add cluster-autoscaler https://kubernetes.github.io/autoscaler
helm repo update

```

Get the helm values and store it into a file

```
helm show values cluster-autoscaler/cluster-autoscaler > autoscaler_values.yaml
```

Change the below setting in the values:

```
vi autoscaler_values.yaml
```

Change the field ClusterName, Namespace

```
autoDiscovery:
  # cloudProviders `aws`, `gce`, `azure`, `magnum`, `clusterapi` and `oci` are supported by auto-discovery at this time
  # AWS: Set tags as described in https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/aws/README.md#auto-discovery-setup

  # autoDiscovery.clusterName -- Enable autodiscovery for `cloudProvider=aws`, for groups matching `autoDiscovery.tags`.
  # autoDiscovery.clusterName -- Enable autodiscovery for `cloudProvider=azure`, using tags defined in https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/azure/README.md#auto-discovery-setup.
  # Enable autodiscovery for `cloudProvider=clusterapi`, for groups matching `autoDiscovery.labels`.
  # Enable autodiscovery for `cloudProvider=gce`, but no MIG tagging required.
  # Enable autodiscovery for `cloudProvider=magnum`, for groups matching `autoDiscovery.roles`.
  clusterName: "live-eks-cluster"

  # autoDiscovery.namespace -- Enable autodiscovery via cluster namespace for for `cloudProvider=clusterapi`
  namespace: "kube-system"

  # autoDiscovery.tags -- ASG tags to match, run through `tpl`.
  tags:
    - k8s.io/cluster-autoscaler/enabled
    - k8s.io/cluster-autoscaler/{{ .Values.autoDiscovery.clusterName }}
  # - kubernetes.io/cluster/{{ .Values.autoDiscovery.clusterName }}

  # autoDiscovery.roles -- Magnum node group roles to match.
  roles:
    - worker
```

```
awsRegion
```



```
# - name: ig02
#   maxSize: 10
#   minSize: 0

# awsAccessKeyID -- AWS access key ID ([if AWS user keys used](https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/aws/README.md))
awsAccessKeyID: ""

# awsRegion -- AWS region (required if 'cloudProvider=aws')
awsRegion: "us-east-1"

# awsSecretAccessKey -- AWS access secret key ([if AWS user keys used](https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/aws/README.md))
awsSecretAccessKey: ""

# azureClientID -- Service Principal ClientID with contributor permission to Cluster and Node ResourceGroup.
# Required if 'cloudProvider=azure'.
```

```
rbac.serviceAccount.create=false
rbac.serviceAccount.name=cluster-autoscaler
```

```
rbac:
  # rbac.create -- If 'true', create and use RBAC resources.
  create: true
  # rbac.pspEnabled -- If 'true', creates and uses RBAC resources required in the cluster with [Pod Security Policies](https://kubernetes.io/docs/concepts/security/pod-security-policies/).
  # Must be used with 'rbac.create' set to 'true'.
  pspEnabled: false
  # rbac.clusterScoped -- if set to false will only provision RBAC to alter resources in the current namespace. Most useful for Cluster-API
  clusterScoped: true
  # rbac.annotations -- Additional annotations to add to RBAC resources (Role/RoleBinding/ClusterRole/ClusterRoleBinding).
  annotations: {}
  serviceAccount:
    # rbac.serviceAccount.annotations -- Additional Service Account annotations.
    annotations: {}
    # rbac.serviceAccount.create -- If 'true' and 'rbac.create' is also true, a Service Account will be created.
    create: false
    # rbac.serviceAccount.name -- The name of the ServiceAccount to use. If not set and create is 'true', a name is generated using the fullname template.
    name: "cluster-autoscaler"
    # rbac.serviceAccount.automountServiceAccountToken -- Automount API credentials for a Service Account.
    automountServiceAccountToken: true
  # rbac.additionalRules -- Additional rules for role/clusterrole
  additionalRules: []
  # rbac.configure -- If 'true', configure RBAC resources.
```

Come to `extraArgs` , and enable them with proper settings

`extraArgs`:

```
balance-similar-node-groups: "true"
skip-nodes-with-system-pods: "false"
scale-down-unneeded-time: "5m"
scale-down-delay-after-add: "2m"
scale-down-utilization-threshold: "0.5"
```

```
extraArgs:
  logtostderr: true
  stderrthreshold: info
  v: 4
  # write-status-configmap: true
  # status-config-map-name: cluster-autoscaler-status
  # leader-elect: true
  # leader-elect-resource-lock: endpoints
  # skip-nodes-with-local-storage: true
  # expander: random
  # scale-down-enabled: true
  balance-similar-node-groups: true
  # min-replica-count: 0
  scale-down-utilization-threshold: 0.5
  # scale-down-non-empty-candidates-count: 30
  # max-node-provision-time: 15m0s
  # scan-interval: 10s
  scale-down-delay-after-add: 2m
  # scale-down-delay-after-delete: 0s
  # scale-down-delay-after-failure: 3m
  scale-down-unneeded-time: 5m
  # node-deletion-delay-timeout: 2m
  # node-deletion-batcher-interval: 0s
  skip-nodes-with-system-pods: false
  # balancing-ignore-label_1: first-label-to-ignore
  # balancing-ignore-label_2: second-label-to-ignore
```



In Kubernetes, scheduling works like this:

- Pods request CPU/memory
- Scheduler tries to place them on nodes
- If no node has capacity → pod goes into Pending state

Now Cluster Autoscaler watches:

“Are there pending pods that cannot be scheduled?”

If yes:

- It checks Auto Scaling Groups in Amazon Elastic Kubernetes Service (EKS)
- Then increases node count

If no:

- It may scale down unused nodes

What these flags actually mean

You added these:

(A) balance-similar-node-groups

```
--balance-similar-node-groups
```

Meaning:

If you have multiple node groups with similar capacity:

- It spreads scaling evenly across them
- Avoids overloading one group

Why needed:

Without this:

- One node group may scale too much
- Others remain idle → inefficient cost distribution

(B) skip-nodes-with-system-pods=false

```
--skip-nodes-with-system-pods=false
```

Meaning:

Allows autoscaler to consider nodes even if they run system pods (like CoreDNS, CNI, metrics-server).

Why needed:

Default behavior is very safe:

- It avoids touching nodes with system workloads

But in real clusters:

- Almost every node has system pods
- So autoscaler becomes too conservative

This flag makes scaling more aggressive.

(C) scale-down-unnneeded-time=5m

```
--scale-down-unnneeded-time=5m
```

Meaning:

Node must be underutilized for 5 minutes before scale down happens.

Why needed:

Prevents:

- Flapping (scale up/down repeatedly)
- Instability during short traffic drops

(D) scale-down-delay-after-add=2m

```
--scale-down-delay-after-add=2m
```

Meaning:

After adding a new node, wait 2 minutes before considering scale down.

Why needed:

- New nodes need time to stabilize
- Pods need time to reschedule

(E) scale-down-utilization-threshold=0.5

```
--scale-down-utilization-threshold=0.5
```

Meaning:

If a node is less than 50% utilized → eligible for scale down.

Why needed:

Controls cost vs stability tradeoff:

- Lower value → aggressive cost saving
- Higher value → safer but more expensive

(F) image version

```
registry.k8s.io/autoscaling/cluster-autoscaler:v1.29.0
```

Why important:

Cluster Autoscaler version must match Kubernetes version.
Mismatch leads to:

- Wrong API calls
- Scaling failures
- CrashLoopBackOff

Change the image tag as per your cluster version (Use version matching your Kubernetes version)

my cluster version is `1.34` so i kept the same

```
image:
# image.repository -- Image repository
repository: registry.k8s.io/autoscaling/cluster-autoscaler
# image.tag -- Image tag
tag: v1.34.0
# image.pullPolicy -- Image pull policy
pullPolicy: IfNotPresent
## Optionally specify an array of imagePullSecrets.
## Secrets must be manually created in the namespace.
## ref: https://kubernetes.io/docs/tasks/configure-pod-container/pull-image-private-registry/
##
# image.pullSecrets -- Image pull secrets
pullSecrets: []
```

Save and exit.

Install:

```
helm upgrade --install cluster-autoscaler cluster-autoscaler/
cluster-autoscaler -f autoscaler_values.yaml -n kube-system
```

```
ubuntu@ip-172-31-38-241:~$ kubectl --namespace=kube-system get pods -l "app.kubernetes.io/name=aws-cluster-autoscaler,app.kubernetes.io/instance=cluster-autoscaler"
NAME                                READY   STATUS    RESTARTS   AGE
cluster-autoscaler-aws-cluster-autoscaler-5666678fc4-ndsp7  1/1     Running   0           14m
ubuntu@ip-172-31-38-241:~$
```

Tag your ASG (Nodegroup's ASG)

If you created the cluster using `eksctl` then it should be added automatically.

If not added then add it manually.

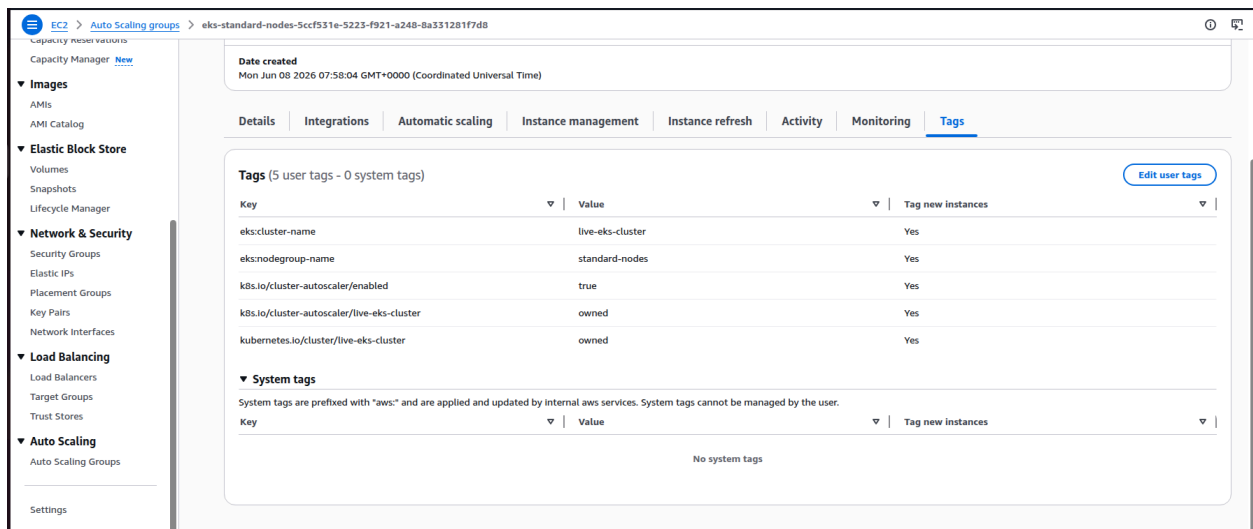
Cluster Autoscaler only manages ASGs with correct tags.

Find ASG:

```
aws autoscaling describe-auto-scaling-groups
```

Add tags:

Key	Value
k8s.io/cluster-autoscaler/enabled	true
k8s.io/cluster-autoscaler/live-eks-cluster	owned



The screenshot shows the AWS Management Console interface for an Auto Scaling Group. The left sidebar contains navigation links for various AWS services. The main content area displays the 'Tags' tab for the Auto Scaling Group 'eks-standard-nodes-Sccf531e-5223-f921-a248-8a331281f7d8'. The 'Tags' section shows 5 user tags and 0 system tags. The tags are listed in a table with columns for Key, Value, and Tag new instances. The system tags section is currently empty, showing 'No system tags'.

Key	Value	Tag new instances
eks:cluster-name	live-eks-cluster	Yes
eks:nodegroup-name	standard-nodes	Yes
k8s.io/cluster-autoscaler/enabled	true	Yes
k8s.io/cluster-autoscaler/live-eks-cluster	owned	Yes
kubernetes.io/cluster/live-eks-cluster	owned	Yes

System tags

System tags are prefixed with "aws:" and are applied and updated by internal aws services. System tags cannot be managed by the user.

Key	Value	Tag new instances
-----	-------	-------------------

No system tags

TEST SCALE UP

Before testing check the nodes available in your cluster.

```
ubuntu@ip-172-31-38-241:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-45-205.ec2.internal      Ready    <none>   6h20m v1.34.8-eks-3385e9b
ubuntu@ip-172-31-38-241:~$
```

Deploy a stress workload:

```
kubectl create deployment stress --image=nginx
kubectl scale deployment stress --replicas=20
```

Check pods:

```
kubectl get pods -w
```

If nodes are insufficient → pods go Pending → autoscaler adds nodes.

Verify Node Scaling

Based on your maximum and minimum size it will scale up and scale down.

```
kubectl get nodes -w
```

or:

```
watch kubectl get nodes
```

You should see new EC2 nodes join automatically.


```

ubuntu@ip-172-31-38-241:~$ kubectl get nodes -w
NAME                                STATUS    ROLES    AGE      VERSION
ip-192-168-45-205.ec2.internal      Ready     <none>   6h22m    v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   0s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   0s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   0s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   1s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   1s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   1s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   1s        v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      NotReady  <none>   11s       v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      Ready     <none>   13s       v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      Ready     <none>   13s       v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      Ready     <none>   16s       v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      Ready     <none>   31s       v1.34.8-eks-3385e9b

```

```

^Cubuntu@ip-172-31-38-241:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE      VERSION
ip-192-168-45-205.ec2.internal      Ready     <none>   6h23m    v1.34.8-eks-3385e9b
ip-192-168-48-212.ec2.internal      Ready     <none>   59s      v1.34.8-eks-3385e9b
ubuntu@ip-172-31-38-241:~$

```

```

^Cubuntu@ip-172-31-38-241:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
stress-7b8fbdcf58-496rv             1/1      Running   0           2m25s
stress-7b8fbdcf58-4j9c9             1/1      Running   0           2m25s
stress-7b8fbdcf58-4v2dd             1/1      Running   0           2m25s
stress-7b8fbdcf58-cvn9h             1/1      Running   0           2m25s
stress-7b8fbdcf58-cwj24             1/1      Running   0           2m25s
stress-7b8fbdcf58-drds6             1/1      Running   0           2m25s
stress-7b8fbdcf58-kdv48             1/1      Running   0           2m25s
stress-7b8fbdcf58-njgc8             1/1      Running   0           2m25s
stress-7b8fbdcf58-nm2mn             1/1      Running   0           2m25s
stress-7b8fbdcf58-ntlrv             1/1      Running   0           2m25s
stress-7b8fbdcf58-q5pcp             1/1      Running   0           2m25s
stress-7b8fbdcf58-s55wm             1/1      Running   0           2m25s
stress-7b8fbdcf58-sqnwq             1/1      Running   0           2m25s
stress-7b8fbdcf58-t482m             1/1      Running   0           2m33s
stress-7b8fbdcf58-tj4vc             1/1      Running   0           2m25s
stress-7b8fbdcf58-ts6qf             1/1      Running   0           2m25s
stress-7b8fbdcf58-v6m4c             1/1      Running   0           2m25s
stress-7b8fbdcf58-wnzgb             1/1      Running   0           2m25s
stress-7b8fbdcf58-zf8h4             1/1      Running   0           2m25s
stress-7b8fbdcf58-zvzzg             1/1      Running   0           2m25s
ubuntu@ip-172-31-38-241:~$

```

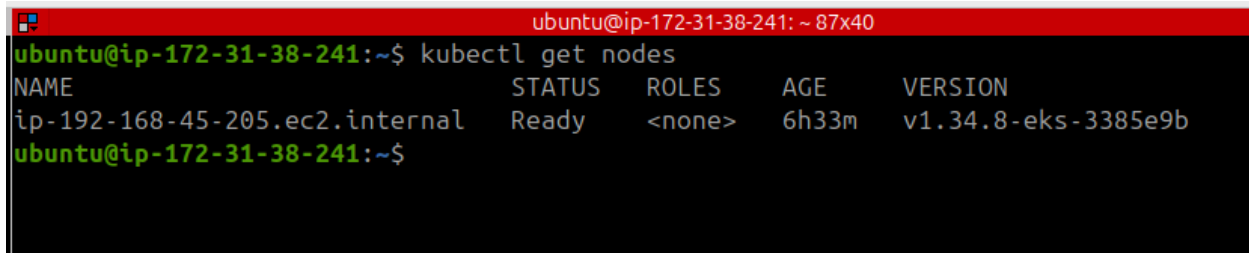
Test Scale Down

After deleting workload:

```
kubectl delete deployment stress
```

Wait ~10–15 minutes → nodes will terminate automatically.

Again it falls back to one node.

A terminal window with a red title bar showing the command 'kubectl get nodes' and its output. The output shows a single node in a 'Ready' state.

```
ubuntu@ip-172-31-38-241: ~ 87x40
ubuntu@ip-172-31-38-241:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-45-205.ec2.internal     Ready    <none>   6h33m v1.34.8-eks-3385e9b
ubuntu@ip-172-31-38-241:~$
```

Debugging Commands

Check logs:

```
kubectl logs -n kube-system deployment/cluster-autoscaler
```

Check events:

```
kubectl get events -A |grep -i scale
```

Summary

Think like this:

```
Pod pending → Cluster Autoscaler detects → calls AWS ASG → new EC2 node → kubelet joins cluster → pod scheduled
```

Clean Up

Delete the cluster:

```
eksctl delete cluster --name live-eks-cluster --region us-east-1
```